ROTEIRO DO PROJETO

DE DESENVOLVIMENTO DE SOLUÇÃO COMPUTACIONAL

Modelo de documento para elaboração e entrega do projeto

Identificação	Preenchimento
Nome do projeto	PORTA A PORTA
Integrantes	LUIZ GUILHERME DE ALMEIDA FERNANDES BRUNO RIBEIRO GARCIA LUÍS EDUARDO BRITO COSTA MELO
Turma	GPE02M80044
Professor	RANYELSON NERES CARVALHO
Data de entrega	15/08/2026

Este documento deve refletir o que foi efetivamente planejado, construído, integrado e testado pelo grupo.

Orientações gerais

- Utilize este roteiro como estrutura mínima obrigatória do documento do projeto.
- Substitua os textos entre colchetes pelas informações reais do projeto.
- Mantenha coerência entre requisitos, diagramas, banco de dados, código-fonte, testes e funcionalidades demonstradas.
- Inclua imagens, diagramas, links e evidências sempre que solicitado.
- Não apresente como concluída uma funcionalidade que não esteja implementada e demonstrável no MVP.
- Caso algum item não se aplique ao projeto, registre a justificativa no próprio tópico.

Importante: documentação, protótipos, telas isoladas, scripts ou trechos de código não caracterizam, por si só, um MVP funcional. Os componentes principais da solução devem estar integrados e permitir a execução do fluxo central do sistema.

1. REQUISITOS

1.1 Visão e escopo

1.1.1 Declaração do problema

O desenvolvimento da solução computacional foi motivado pela desorganização e ineficiência no comércio informal praticado dentro de condomínios residenciais. Atualmente, a comercialização de produtos e serviços é realizada de forma adaptada através de grupos de mensagens instantâneas, como o WhatsApp. Esse formato atual gera séria poluição nos canais de comunicação do condomínio levando à perda de avisos importantes e sobrecarregando a gestão do síndico, além de provocar erros em pedidos e falta de clareza nos preços pela ausência de um catálogo estruturado. Essa rotina impacta negativamente tanto os moradores vendedores quanto os compradores. Uma solução computacional contribui diretamente para minimizar esse cenário ao centralizar as ofertas em uma vitrine digital segura e exclusiva, garantindo a automação do fluxo de pedidos vinculados à unidade do morador e despoluindo os canais oficiais de comunicação.

1.1.2 Visão da solução

Objetivo geral da solução:

Prover uma vitrine digital segura e exclusiva para a comunidade do condomínio, organizando o comércio interno e automatizando a gestão de pedidos entre moradores vizinhos.

Público-alvo:

Moradores de condomínios residenciais (compradores e vendedores/empreendedores) e a administração do condomínio (síndicos e gestão).

Principais benefícios esperados:

- Centralização e Organização das Ofertas: Disponibilização de uma vitrine digital padronizada e exclusiva com catálogo fixo, fotos, preços e carrinho de compras.
- Automação e Precisão na Gestão de Pedidos: Acompanhamento eficiente do fluxo e status das solicitações, eliminando erros no pedido e vinculando cada compra diretamente à unidade/bloco do morador.
- Despoluição dos Canais Oficiais de Comunicação: Redução drástica do envio de anúncios em grupos de WhatsApp do condomínio, evitando a perda de mensagens importantes da comunidade e reduzindo a sobrecarga da administração/síndico.

1.1.3 Escopo

Necessidade identificada	Funcionalidade proposta
Ter um catálogo fixo com fotos e preços	Vitrine Digital
Realizar e vincular pedidos à unidade do morador	Carrinho de Compras e Finalização por Unidade
Notificar novos pedidos e acompanhar entregas	Gerenciador de Status do Pedido
Garantir acesso restrito e seguro aos moradores	Autenticação de Usuários e Isolamento por Condomínio

Itens incluídos no escopo

- Criação de Vitrine Digital: Cadastro de produtos por moradores empreendedores com fotos, descrição e preços fixos.
- Gestão de Carrinho de Compras: Seleção de itens e fechamento de pedidos vinculados obrigatoriamente à identificação da unidade (bloco/apartamento) do morador comprador.
- Gerenciador de Status do Pedido: Painel para vendedores e compradores acompanharem o

andamento, aceite e entrega dos pedidos.

- Notificações internas no aplicativo: Avisos exibidos dentro do próprio app sobre a chegada de novos pedidos e atualizações de status, consultados quando o usuário abre ou atualiza o aplicativo.
- Participação em mais de um condomínio: o morador pode pedir para participar de outro condomínio informando a unidade; o síndico daquele condomínio aprova (definindo se a pessoa só compra ou também vende) ou recusa. Com a aprovação, o morador troca o condomínio exibido na vitrine.
- Painel de vendas: resumo para o vendedor com o total vendido, os pedidos em andamento, os pedidos por status e os produtos mais vendidos.
- Moderação de usuários: painel do síndico com todos os usuários do condomínio, os que chegaram nos últimos 30 dias e as ações de advertência, suspensão por prazo e banimento, sempre com motivo.
- Foto de perfil: cada usuário pode adicionar, trocar ou remover a própria foto.

Itens fora do escopo

- Processamento de Pagamentos Online: Não inclui integração com gateways de pagamento (cartão de crédito, Pix automático, etc.). Toda a transação financeira ocorre diretamente entre os moradores (modo offline).
- Logística de Entrega Externa: O sistema não realiza gestão de frete, entregadores externos ou rastreamento de transporte fora do condomínio.
- Notificações Push: O sistema não envia alertas via push (Expo Push Notifications/FCM), pois esse recurso exige build nativo do aplicativo e integração com serviço externo. Os avisos ficam restritos às notificações internas do aplicativo.

1.2 Especificação de requisitos

1.2.1 Histórias de usuário

ID	História de usuário	Prioridade
HU01	Como Morador Comprador, eu quero consultar a vitrine digital do condomínio filtrada por categorias , para visualizar de forma rápida os produtos e serviços ativos oferecidos exclusivamente pelos meus vizinhos	Alta
HU02	Como Morador Comprador, eu quero realizar um pedido selecionando uma janela de horário para entrega ou retirada , para agendar o recebimento da mercadoria de forma organizada e compatível com a minha rotina	Alta
HU03	Como Morador Vendedor, eu quero cadastrar e manter anúncios de produtos respeitando o limite máximo de 15 itens ativos , para divulgar meu catálogo comercial de forma limpa e organizada dentro da comunidade do condomínio	Alta
HU04	Como Morador Vendedor, eu quero gerenciar o painel de entregas e atualizar o status do pedido em tempo real, gerando notificações internas no aplicativo, para otimizar minha fila de produção e manter o comprador informado sobre o progresso da entrega	Média

1.2.2 Regras de negócio

ID	Regra de negócio
RN01	Limite de Anúncios Ativos por Vendedor: O morador cadastrado como vendedor pode manter, no máximo, 15 itens ativos simultaneamente em sua vitrine digital. Caso atinja essa marca, o sistema bloqueia a inserção de novos produtos até que um anúncio antigo seja desativado, pausado ou excluído.
RN02	Moderação de Conteúdo Gerado por Usuários (Denúncias e Avaliações): Para preservar a confiança da comunidade, toda ação de moderação sobre conteúdo sensível gerado por usuários segue o princípio de que apenas o síndico tem poder de decisão final. (a) Denúncias e quarentena automática: um anúncio que receber denúncias de 3 CPFs distintos é automaticamente movido para o estado "Quarentena" (oculto da vitrine), gerando alerta para análise humana do síndico, que decide manter a remoção, restaurar o anúncio ou aplicar outra penalidade. O anúncio em quarentena fica visível apenas para o síndico e para o próprio vendedor, que vê em destaque apenas o motivo informado pelo síndico (na suspensão automática, "Muitas denúncias: produto sob análise do síndico"); os motivos escritos pelos denunciantes são vistos somente pelo síndico, sem identificação de quem denunciou; somente o síndico pode retirá-lo da quarentena. (b) Avaliações e indelebilidade: cada produto de um pedido entregue pode ser avaliado uma vez pelo comprador (nota 1-5 e comentário); a média e os comentários aparecem no anúncio, sem identificar quem avaliou. As avaliações são definitivas e não podem ser editadas pelo autor após o envio; apenas o síndico pode removê-las, mediante solicitação do vendedor afetado ou de ofício, quando identificar conteúdo abusivo ou de má-fé. (c) Privacidade do denunciante/avaliador: a denúncia é sempre exibida sem o nome, a unidade ou qualquer outro dado que identifique o denunciante, tanto para os demais moradores e para o vendedor denunciado quanto para o síndico, que visualiza na fila de moderação apenas o anúncio, o motivo e a data da denúncia. Da mesma forma, a identidade de quem avalia não é exposta a outros moradores nem ao vendedor avaliado. O sistema mantém internamente o vínculo com o autor (CPF/ID) somente para permitir a contagem de CPFs distintos e impedir denúncias duplicadas, sem exibi-lo em nenhuma tela. As decisões de moderação do síndico (quarentena, restauração de anúncio e remoção de avaliação) são registradas em log de auditoria, com a respectiva justificativa.
RN03	Restrição de Transação Financeira e Logística Integrada: O sistema opera estritamente sob escopo informacional e de gestão interna, não realizando processamento de pagamentos online (como cartões ou Pix integrados) e não oferecendo suporte a frete ou logística externa . Todo o pagamento e entrega devem ocorrer de forma direta e offline entre os vizinhos.

1.2.3 Requisitos não funcionais

ID	Categoria	Requisito não funcional
RNF01	Usabilidade	O sistema deve manter o usuário informado sobre novos pedidos e mudanças de status por meio de notificações internas no aplicativo, sem depender de serviço externo de push.
RNF02	Segurança	O acesso deve ser rigidamente restrito à comunidade local, exigindo a vinculação obrigatória de todo pedido à identificação da unidade do morador (Ex: Bloco/Apartamento). Toda a comunicação de dados deve trafegar sob o protocolo HTTPS para garantir criptografia básica, e o banco de dados deve utilizar mecanismos de Multitenancy (isolamento lógico de dados) por meio da chave estrangeira condominio_id em todas as entidades do sistema.
RNF03	Compatibilidade	O sistema deve garantir a persistência integral de todos os dados de catálogos e pedidos diretamente no banco de dados relacional, mitigando quaisquer chances de perda de dados ou inconsistência em cenários de quedas de energia ou do servidor.
RNF04	Escalabilidade	A infraestrutura inicial do servidor em nuvem (com custo estimado de R$ 50,00/mês) deve ser dimensionada para suportar a operação e o tráfego simultâneos de até 20 condomínios de pequeno porte .
RNF05	Integridade e Desempenho	O sistema deve impor limites estritos no upload de mídia para os anúncios, recusando arquivos que não sejam do formato de imagem ou que excedam o tamanho máximo de 5MB por foto.

2. PROJETO DE SOFTWARE

2.1 Projeto de interface com usuário

Tela 1: Tela de Login

Item	Descrição
Objetivo da tela	Formulário de autenticação dos usuários.
Usuário que utilizará	Compradores e Vendedores.
Principais funcionalidades	Realiza a autenticação dos usuários no sistema.
Protótipo	https://freeimage.host/i/CZYQvxp

Tela 2: Tela de Pedidos

Item	Descrição
Objetivo da tela	Exibir ao Usuário os pedidos feitos no sistema.
Usuário que utilizará	Compradores.
Principais funcionalidades	Exibe informações e status dos pedidos feitos.
Protótipo	https://freeimage.host/i/img-pedidos.CZYtDl9

Tela 3: Tela de Perfil

Item	Descrição
Objetivo da tela	Exibir ao Usuário o perfil no aplicativo.
Usuário que utilizará	Compradores e Vendedores.
Principais funcionalidades	Exibe informações sobre um Usuário.
Protótipo	https://freeimage.host/i/CZYDtXR

Tela 4: Tela Principal

Item	Descrição
Objetivo da tela	Tela principal do Aplicativo.
Usuário que utilizará	Compradores e Vendedores.
Principais funcionalidades	Exibe informações sobre produtos da vizinhança e menus de navegação.
Protótipo	https://freeimage.host/i/CZYb5LQ

2.2 Arquitetura lógica

2.2.1 Estilo arquitetural

Item	Descrição
Arquitetura adotada	Cliente-servidor com API REST; backend organizado em módulos de rotas por recurso, que concentram a validação das entradas e as regras de negócio e acessam o banco diretamente pelo ORM, com middlewares compartilhados de autenticação, autorização por perfil e tratamento de erros.
Justificativa da escolha	O app mobile (cliente) consome uma API REST central, permitindo que múltiplos condomínios compartilhem o mesmo backend com isolamento lógico por condominio_id (RNF02). A organização em módulos de rotas com acesso direto ao ORM mantém cada regra de negócio (RN01-RN03) junto ao recurso que ela afeta e reduz a quantidade de camadas e de código para o porte do MVP, enquanto os middlewares evitam a repetição de autenticação e tratamento de erros. Como o servidor não mantém estado de sessão, é possível escalá-lo horizontalmente conforme exigido pelo RNF04 (até 20 condomínios).

2.2.2 Componentes da aplicação

Componente/Módulo	Responsabilidade
Módulo de Autenticação	Login, cadastro, foto de perfil e vinculação do usuário à unidade e ao condomínio (RNF02). Os perfis são hierárquicos: o vendedor tem também as permissões do comprador, e o síndico as do vendedor e do comprador. Usuário suspenso ou banido não acessa o condomínio enquanto durar a sanção.
Módulo de Vitrine/Catálogo	CRUD de anúncios com uma foto por produto armazenada no servidor (RNF05), controle do limite de 15 itens ativos (RN01) e busca por categoria (HU01).
Módulo de Pedidos	Carrinho, checkout vinculado à unidade, escolha de dia e período de entrega (HU02), status do pedido (HU04) e painel de vendas do vendedor. Nenhum usuário compra o próprio produto.
Módulo de Moderação	Denúncias, quarentena automática e avaliações por produto (média e comentários anônimos exibidos no anúncio), conforme RN02.
Módulo de Condomínios	Pedidos de participação em outros condomínios, aprovação ou recusa pelo síndico de cada condomínio e troca do condomínio ativo. Fora do condomínio de origem valem a unidade e o perfil aprovados (RNF02).
Módulo de Notificações	Registro e exibição de avisos internos no aplicativo (tela de avisos com contador de não lidos): novo pedido para o vendedor, mudança de status para o comprador, quarentena automática para o síndico e o vendedor decisão do síndico e nova avaliação para o vendedor (RNF01).
Painel do Síndico	Tela de moderação em quatro abas: fila de denúncias agrupada por anúncio, sem identificação do denunciante, decisão sobre quarentena e avaliações, painel de usuários do condomínio (novos usuários, pedidos de participação, advertência, suspensão, banimento e reativação) e histórico das decisões de moderação, lido do log de auditoria.

2.2.3 Diagramas

Diagrama de Classes

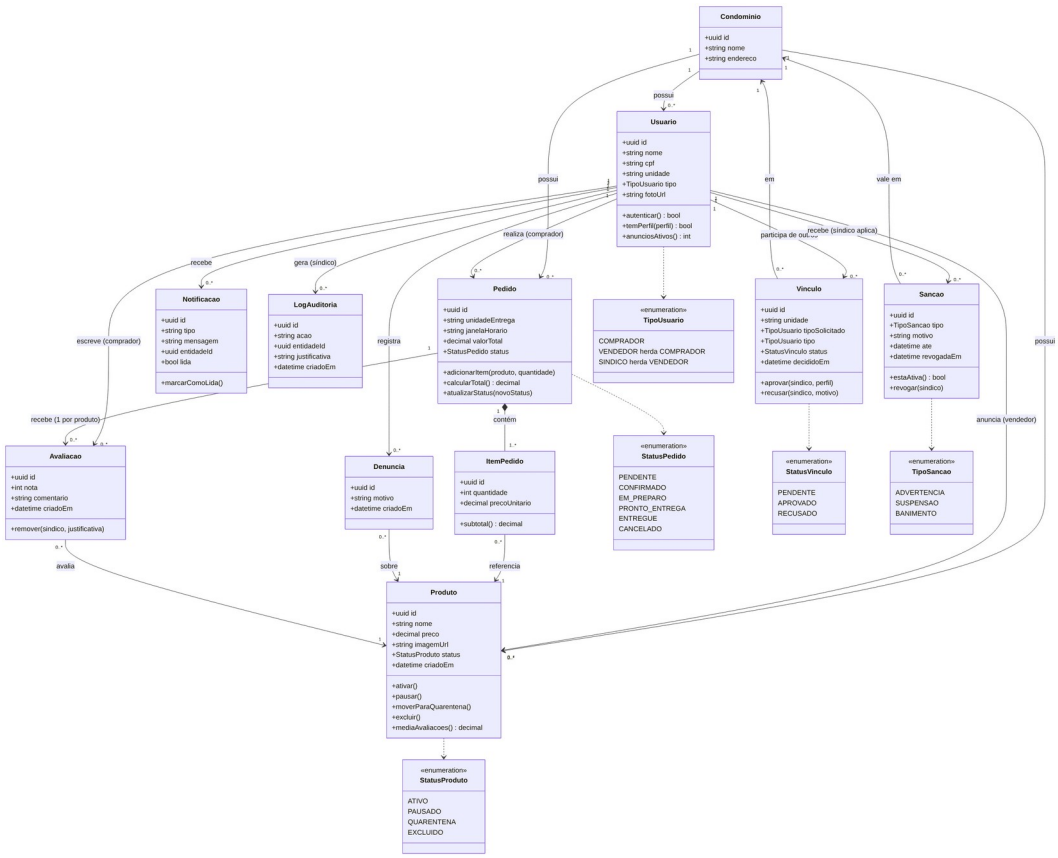
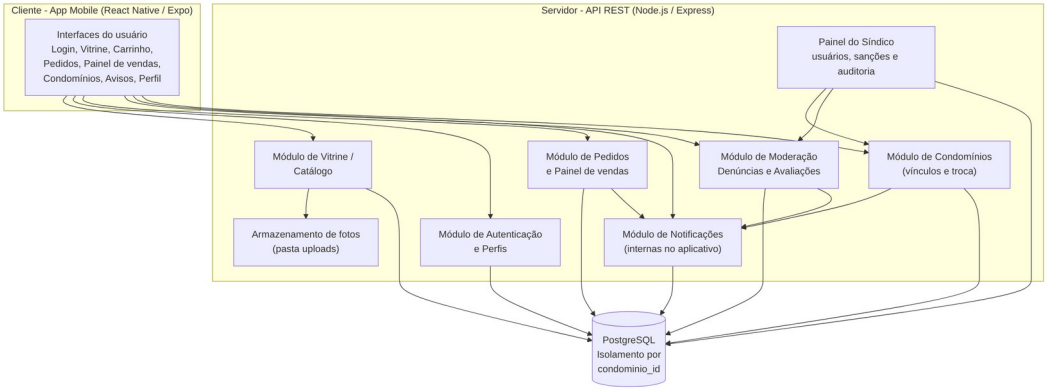


Diagrama de Pacotes ou Componentes



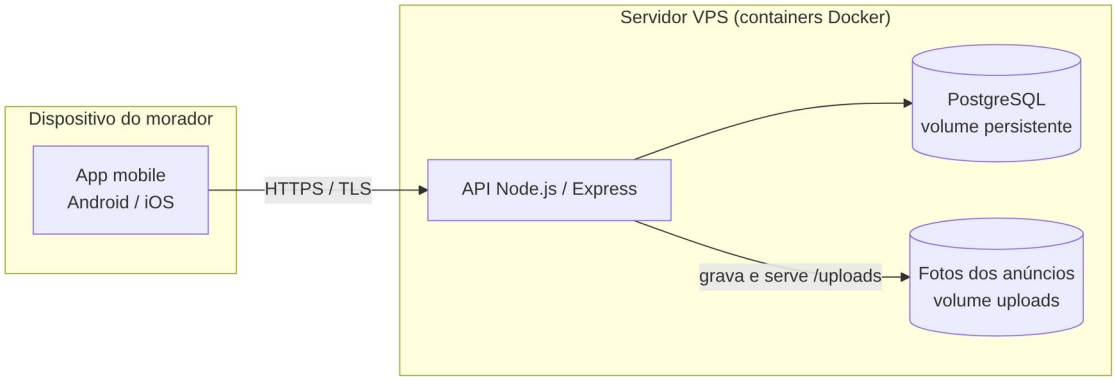
2.2.4 Tecnologias utilizadas

Categoria	Tecnologia
Linguagem de programação	JavaScript / TypeScript
Front-end	React Native (Expo)
Back-end	Node.js + Express
Banco de dados	PostgreSQL
Frameworks	Express, Prisma (ORM)
APIs/Serviços externos	Não se aplica: o sistema não depende de serviços externos. As fotos dos anúncios ficam armazenadas no próprio servidor.
Controle de versão	Git + GitHub

2.3 Arquitetura física

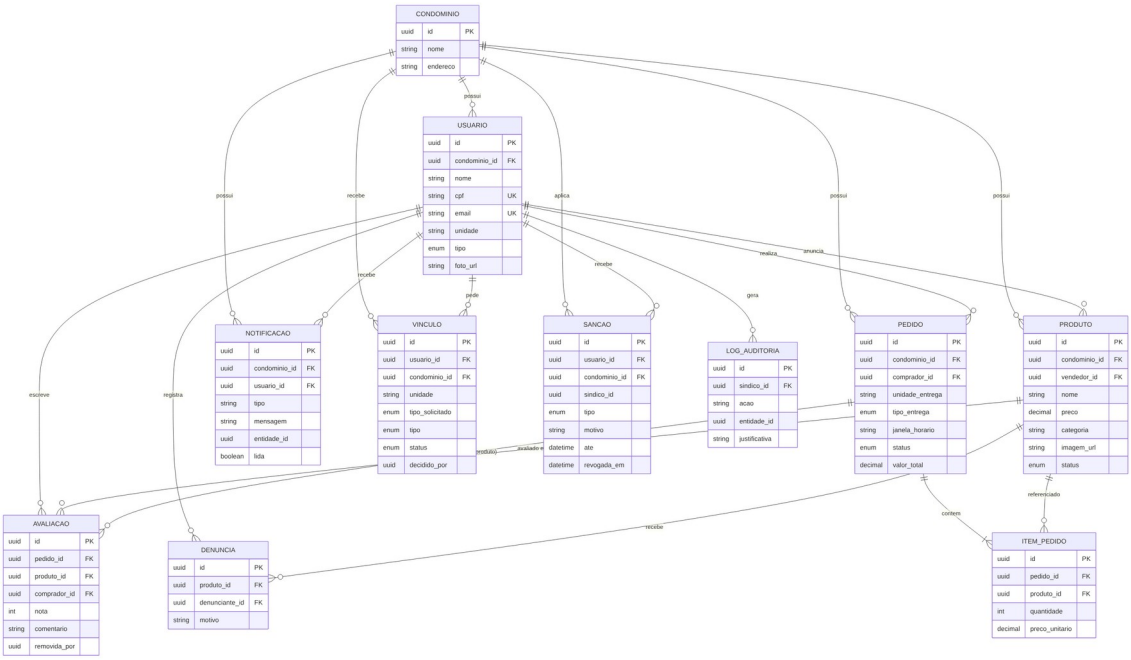
Recurso	Descrição
Cliente	App mobile React Native (Expo), instalado no smartphone do morador (Android/iOS).
Servidor	Aplicação Node.js/Express executada em container Docker em um servidor VPS.
Banco de dados	Instância PostgreSQL executada em container no mesmo servidor VPS, com volume persistente.
Rede/Internet	Comunicação via HTTPS (TLS) entre app e API, conforme RNF02.
Serviço de hospedagem	Servidor VPS em provedor de nuvem (~R$ 50/mês), dimensionado para até 20 condomínios (RNF04).
Outros recursos	Pasta de uploads no servidor, em volume persistente, com as fotos dos anúncios (uma por produto, até 5 MB - RNF05), servidas pela própria API.

Diagrama de implantação



2.4 Projeto de banco de dados

2.4.1 Modelo Entidade-Relacionamento



2.4.2 Descrição das tabelas

Tabela	Finalidade
condominio	Isola cada condomínio (multitenancy — RNF02).
usuario	Moradores e síndico, vinculados à unidade e ao condomínio, com foto de perfil opcional. O tipo define o perfil, com herança de permissões (COMPRADOR, VENDEDOR e SINDICO).
produto	Anúncios da vitrine, com limite de 15 ativos por vendedor (RN01) e o caminho da foto armazenada no servidor.
pedido	Pedidos vinculados à unidade do comprador (HU02).
item_pedido	Itens de cada pedido (relação N:N entre pedido e produto).
avaliacao	Avaliações de cada produto comprado (uma por produto do pedido entregue), anônimas e imutáveis pelo autor (RN02).
denuncia	Denúncias de anúncios, usadas para disparar a quarentena (RN02).
notificacao	Avisos internos exibidos no aplicativo, com mensagem, referência ao pedido ou anúncio e indicação de leitura (RNF01).
vinculo	Participação de um usuário em um condomínio além do seu condomínio de origem, com unidade, perfil aprovado e situação do pedido (pendente, aprovado ou recusado) (RNF02).
sancao	Advertências, suspensões (com data de fim) e banimentos aplicados pelo síndico a um usuário em um condomínio, com motivo e eventual reativação.
log_auditoria	Registro das ações de moderação do síndico (quarentena, restauração de anúncio, remoção de avaliação, decisão sobre pedidos de participação e sanções a usuários), com justificativa (RN02).

2.4.3 Estrutura das tabelas

Os identificadores (id e chaves estrangeiras) são do tipo UUID, gerados pela aplicação, o que evita expor a sequência de registros e facilita a combinação de dados entre condomínios.

Tabela: produto

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
condominio_id	UUID	FK -> condominio	Isolamento multitenant
vendedor_id	UUID	FK -> usuario, NOT NULL	Dono do anuncio
nome	VARCHAR	NOT NULL	Nome do produto
preco	DECIMAL	NOT NULL	Preco fixo
status	VARCHAR	NOT NULL	ativo / pausado / quarentena / excluido (RN02)
imagem_url	VARCHAR	NULL	Caminho da foto no servidor (RNF05)
criado_em	TIMESTAMP	NOT NULL	Data de cadastro

Tabela: denuncia

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
produto_id	UUID	FK -> produto, NOT NULL	Anuncio denunciado
denunciante_id	UUID	FK -> usuario, NOT NULL	Autor da denuncia (uso interno, nao exibido - RN02)
motivo	VARCHAR	NOT NULL	Categoria da denuncia
criado_em	TIMESTAMP	NOT NULL	Data da denuncia
-	-	UNIQUE(produto_id, denunciante_id)	Impede denuncia duplicada do mesmo usuario (RN02)

Tabela: avaliacao

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
produto_id	UUID	FK -> produto, NOT NULL	Produto avaliado
pedido_id	UUID	FK -> pedido, NOT NULL	Pedido avaliado
comprador_id	UUID	FK -> usuario, NOT NULL	Autor da avaliacao
nota	INTEGER	NOT NULL, CHECK 1-5	Nota de 1 a 5
comentario	TEXT	-	Comentario livre
removida_por	UUID	FK -> usuario (nullable)	Sindico que removeu, se houver (RN02)
criado_em	TIMESTAMP	NOT NULL	Nao editavel apos insercao (RN02)
-	-	UNIQUE(pedido_id, produto_id)	Uma avaliacao por produto de cada pedido (RN02)

Tabela: log_auditoria

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
sindico_id	UUID	FK -> usuario, NOT NULL	Sindico que realizou a acao
acao	VARCHAR	NOT NULL	Ex: aplicou quarentena, restaurou anuncio, removeu avaliacao, decidiu participacao, advertiu, suspendeu, baniu ou reativou usuario
entidade_id	UUID	NOT NULL	Anuncio, avaliacao ou vinculo moderado
justificativa	TEXT	NOT NULL	Motivo da decisao de moderacao (RN02)
criado_em	TIMESTAMP	NOT NULL	Data/hora da acao

Tabela: vinculo

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
usuario_id	UUID	FK -> usuario, NOT NULL	Quem pediu para participar
condominio_id	UUID	FK -> condominio, NOT NULL	Condominio em que quer participar
unidade	VARCHAR	NOT NULL	Unidade da pessoa nesse condominio
tipo_solicitado	VARCHAR	NOT NULL	Perfil pedido: comprador ou vendedor
tipo	VARCHAR	NULL	Perfil aprovado pelo sindico
status	VARCHAR	NOT NULL	pendente / aprovado / recusado (RNF02)
decidido_por	UUID	NULL	Sindico que decidiu o pedido
decidido_em	TIMESTAMP	NULL	Data da decisao
-	-	UNIQUE(usuario_id, condominio_id)	Um vinculo por usuario em cada condominio

Tabela: sancao

Campo	Tipo	Restrição	Descrição
id	UUID	PK	Identificador
usuario_id	UUID	FK -> usuario, NOT NULL	Usuario que recebeu a sancao
condominio_id	UUID	FK -> condominio, NOT NULL	Condominio em que a sancao vale
sindico_id	UUID	NOT NULL	Sindico que aplicou
tipo	VARCHAR	NOT NULL	advertencia / suspensao / banimento
motivo	TEXT	NOT NULL	Motivo informado ao usuario
ate	TIMESTAMP	NULL	Fim da suspensao (vazio no banimento e na advertencia)
revogada_em	TIMESTAMP	NULL	Quando o sindico reativou o usuario
revogada_por	UUID	NULL	Sindico que reativou
criado_em	TIMESTAMP	NOT NULL	Data da sancao

3. CONSTRUÇÃO

3.1 Código fonte

Item	Informação
Repositório do projeto	https://github.com/luizgaf/porta-a-porta
Linguagem principal	TypeScript, no front-end e no back-end.
Frameworks e bibliotecas utilizadas	Front-end: React Native com Expo, Expo Router (navegação), Zustand (estado), AsyncStorage (persistência local), Axios (HTTP), Expo Image Picker (foto do produto) e Expo Font com as fontes Fraunces e DM Sans. Back-end: Node.js com Express, Prisma ORM, Zod (validação), JSON Web Token e bcryptjs (autenticação), Helmet, CORS e Morgan. Testes de unidade com Jest e ts-jest. Banco de dados: PostgreSQL, executado via Docker Compose.

3.1.1 Organização do código

O repositório é único e está dividido em três pastas principais: backend (API REST), frontend (aplicativo móvel) e docker (banco de dados local). O back-end é organizado em módulos de rotas por recurso, e o front-end segue o roteamento por arquivos do Expo Router, com uma tela por arquivo.

Pasta/arquivo	Conteúdo e responsabilidade
backend/src/index.ts	Inicialização do servidor Express: middlewares globais (Helmet, CORS e Morgan), verificação de saúde (/health), registro das rotas e conexão com o banco.
backend/src/routes/	Um módulo de rotas por recurso (auth, condominio, usuario, produto, pedido, denuncia, avaliacao, auditoria, notificacao e vinculo; usuario inclui a foto de perfil, o painel do síndico e as sanções), com validação das entradas, regras de negócio e acesso ao banco. O arquivo index.ts registra todos sob o prefixo /api.
backend/src/middleware/	auth.ts (autenticação por token, condomínio ativo e vínculo aprovado, e autorização por perfil com a hierarquia de perfis) e errorHandler.ts (classe de erro da aplicação e tratamento centralizado de erros).
backend/src/services/	Funções compartilhadas entre as rotas: notificacoes.ts (gravação dos avisos internos), imagens.ts (leitura da foto enviada em formulário ou direto, validação pelo conteúdo, gravação e remoção), sessao.ts (token com o condomínio ativo), unidades.ts (unidade do vínculo de quem participa de outro condomínio), quarentena.ts (motivo da quarentena e, só para o síndico, os motivos das denúncias, sem identificação), avaliacoes.ts (média e quantidade de avaliações de cada produto) e sancoes.ts (suspensão ou banimento em vigor e a mensagem mostrada ao usuário).
backend/prisma/	schema.prisma (modelos e enums), migrations (scripts SQL de criação e atualização do banco), seed.ts (dados iniciais) e seed-images (fotos padrão dos produtos de exemplo).
backend/uploads/	Fotos dos produtos enviadas pelos vendedores, servidas pela API no endereço /uploads (não versionada).
frontend/app/	Telas do aplicativo, uma por arquivo: (auth) para boas-vindas, login, cadastro e seleção de perfil e de condomínio; (tabs) para vitrine, busca, carrinho, pedidos, painel de vendas, anúncios do vendedor, moderação e perfil; condominios para trocar de condomínio e pedir participação; product, order, notifications e settings para cadastro e detalhe de produto, detalhe de pedido, avisos e configurações.
frontend/components/	Componentes de interface reutilizáveis (Button, Input, Card, Avatar com foto, CategoryChip, DoorTag, ProductCard, ProductTile, ProductImage, AvisoQuarentena e Estrelas), componentes das telas de acesso, as avaliações do produto (avaliacoes/AvaliacoesDoProduto), os componentes da moderação, um por aba (FilaDenuncias, AvaliacoesModeracao, PainelUsuarios com PedidosParticipacao e HistoricoModeracao) e o provedor de autenticação.
frontend/hooks/	useAuth (sessão e perfil do usuário) e useApi (chamadas à API agrupadas por recurso).
frontend/store/	Estado global com Zustand, persistido no dispositivo: authStore (sessão) e cartStore (carrinho).

Pasta/arquivo	Conteúdo e responsabilidade
frontend/utils/	api.ts (cliente HTTP com Axios, que envia o token em cada requisição e trata as respostas de erro) e funções auxiliares de formatação, seleção e envio de fotos e paginação das listas.
frontend/constants/ e frontend/types/	Constantes, rótulos e tokens de design (cores, tipografia e espaçamentos); tipos TypeScript compartilhados entre as telas.
docker/	docker-compose.yml do PostgreSQL, script de inicialização do banco e run.sh, que sobe e para o container e prepara o banco (setup: migrações e dados iniciais; reset: recria o banco do zero).
adb/	Scripts tunnel.sh e tunnel.cmd, que redirecionam as portas da API (3000) e do Expo (8081) para um dispositivo Android conectado por USB, para testar o aplicativo em aparelho físico (com o comando npm run start:local).
TUTORIAL.md	Guia de execução do projeto e contas de teste.

3.1.2 Padrões de qualidade adotados

- Padronização dos nomes de variáveis: variáveis e funções em camelCase (condominioId, valorTotal); componentes, tipos e modelos em PascalCase (ProductCard, Usuario); valores de enum em maiúsculas (ATIVO, PENDENTE); tabelas e colunas do banco em snake_case (condominio_id, criado_em). Os nomes de domínio são escritos em português.
- Organização em classes, módulos ou componentes: no back-end, um módulo de rotas por recurso; no front-end, uma tela por arquivo, componentes de interface em components/ui e estado global em stores separados.
- Separação de responsabilidades: autenticação, autorização por perfil e tratamento de erros ficam em middlewares; a validação das entradas é feita por esquemas declarados no início de cada módulo de rotas; a persistência passa pelo ORM, e as funções usadas por mais de um módulo (avisos e fotos) ficam em services. No aplicativo, as telas não acessam a API diretamente, e sim por meio do hook useApi.
- Reutilização de código: funções e classes compartilhadas por todas as rotas (asyncHandler, AppError, authMiddleware e requireRole); componentes de interface, tokens de design e tipos reutilizados entre as telas.
- Tratamento de erros: tratamento centralizado na API, que devolve respostas padronizadas com mensagem, código do erro e o status HTTP adequado (400 para validação, 401 e 403 para acesso, 404 para registro inexistente e 409 para duplicidade). No aplicativo, as chamadas são feitas com try/catch e os erros das ações do usuário são exibidos em alertas.
- Comentários quando necessários: comentários curtos em português identificam cada endpoint e marcam os pontos em que uma regra de negócio é aplicada (por exemplo, a RN01 no cadastro e na ativação de produtos).
- Controle de versão: Git, com repositório público no GitHub e histórico no ramo master; as alterações são desenvolvidas em ramos de trabalho e integradas por pull request. A estrutura do banco também é versionada, por meio das migrações do Prisma. As configurações sensíveis (arquivo .env) não são versionadas; o repositório traz apenas o modelo .env.example.
- Organização de commits: mensagens descritivas, em sua maioria com prefixo de tipo (feat, fix, chore e docs), indicando a natureza de cada alteração.
- Tipagem estática: TypeScript em modo strict no front-end e no back-end, com tipos compartilhados para as entidades do sistema.

3.2 Teste de unidade

Os testes de unidade foram implementados com Jest no back-end (pasta backend/tests), com o banco de dados simulado, e executados com o comando npm test (Jest). Os testes TU19 a TU32 cobrem as funcionalidades acrescentadas depois da primeira rodada (avisos internos, remoção de avaliação pelo síndico, foto do produto e hierarquia de perfis). Os testes TU33 a TU40 cobrem a participação em outros condomínios e o painel de vendas, os testes TU41 a TU45, a visibilidade e o motivo dos anúncios em quarentena, os testes TU46 a TU53, o envio de fotos, a foto de perfil e o painel de usuários com as sanções, e os testes TU54 a TU58, a avaliação por produto, a fila de moderação agrupada por anúncio e o histórico de moderação. O TU18, referente ao carrinho no aplicativo, ainda não foi implementado.

ID	Unidade testada	Cenário	Resultado esperado	Resultado obtido
TU01	Criação de produto (POST /produtos)	Vendedor com 15 produtos ativos tenta cadastrar mais um (RN01).	Requisição recusada com status 400 e código LIMIT_EXCEEDED; nenhum produto é criado.	Sucesso
TU02	Alteração de status do produto (PATCH /produtos/:id/status)	Vendedor com 15 produtos ativos tenta reativar um produto pausado (RN01).	Status 400 e código LIMIT_EXCEEDED; o produto permanece pausado.	Sucesso
TU03	Alteração de status do produto (PATCH /produtos/:id/status)	Vendedor tenta mover o próprio produto para quarentena.	Status 403 e código FORBIDDEN; apenas o síndico altera para esse status.	Sucesso
TU04	Criação de pedido (POST /pedidos)	Comprador envia pedido com dois produtos e quantidades diferentes (HU02).	Pedido criado (201) com status PENDENTE; valor total igual à soma de preço x quantidade, calculada no servidor; preço unitário gravado em cada item.	Sucesso
TU05	Criação de pedido (POST /pedidos)	Pedido inclui um produto pausado ou de outro condomínio.	Status 400 e código PRODUTO_INVALIDO; nenhum pedido é criado.	Sucesso
TU06	Atualização de status do pedido (PATCH /pedidos/:id/status)	Vendedor avança o pedido de PENDENTE para CONFIRMADO (HU04).	Status 200; o pedido passa a CONFIRMADO.	Sucesso
TU07	Atualização de status do pedido (PATCH /pedidos/:id/status)	Vendedor tenta passar o pedido de PENDENTE direto para ENTREGUE.	Status 400 e código INVALID_TRANSITION; o status não muda.	Sucesso
TU08	Atualização de status do pedido (PATCH /pedidos/:id/status)	Comprador tenta confirmar o próprio pedido.	Status 403 e código FORBIDDEN; o comprador só pode cancelar.	Sucesso
TU09	Registro de denúncia (POST /denuncias)	Três usuários distintos denunciam o mesmo anúncio (RN02).	Após a segunda denúncia o produto continua ATIVO; após a terceira, passa a QUARENTENA.	Sucesso
TU10	Registro de denúncia (POST /denuncias)	O mesmo usuário denuncia o mesmo anúncio duas vezes.	Status 409 e código DUPLICATE_DENUNCIA; a contagem de denúncias não aumenta.	Sucesso
TU11	Listagem de denúncias (GET /denuncias)	Síndico consulta a fila de moderação (RN02).	A resposta não contém nome, unidade ou identificador do denunciante.	Sucesso
TU12	Criação de avaliação (POST /avaliacoes)	Comprador tenta avaliar um pedido que ainda não foi entregue.	Status 404 e código PEDIDO_NOT_ELIGIBLE; nenhuma avaliação é criada.	Sucesso
TU13	Edição de avaliação (PUT /avaliacoes/:id)	Autor tenta alterar a nota depois do envio (RN02).	Status 403 e código AVALIACAO_IMUTAVEL; nota e comentário permanecem inalterados.	Sucesso
TU14	Middleware de autenticação (authMiddleware)	Requisição sem token e requisição com token inválido.	Status 401 com os códigos UNAUTHORIZED e INVALID_TOKEN; a rota protegida não é executada.	Sucesso
TU15	Middleware de autorização (requireRole)	Comprador acessa rota restrita ao síndico.	Status 403 e código FORBIDDEN.	Sucesso

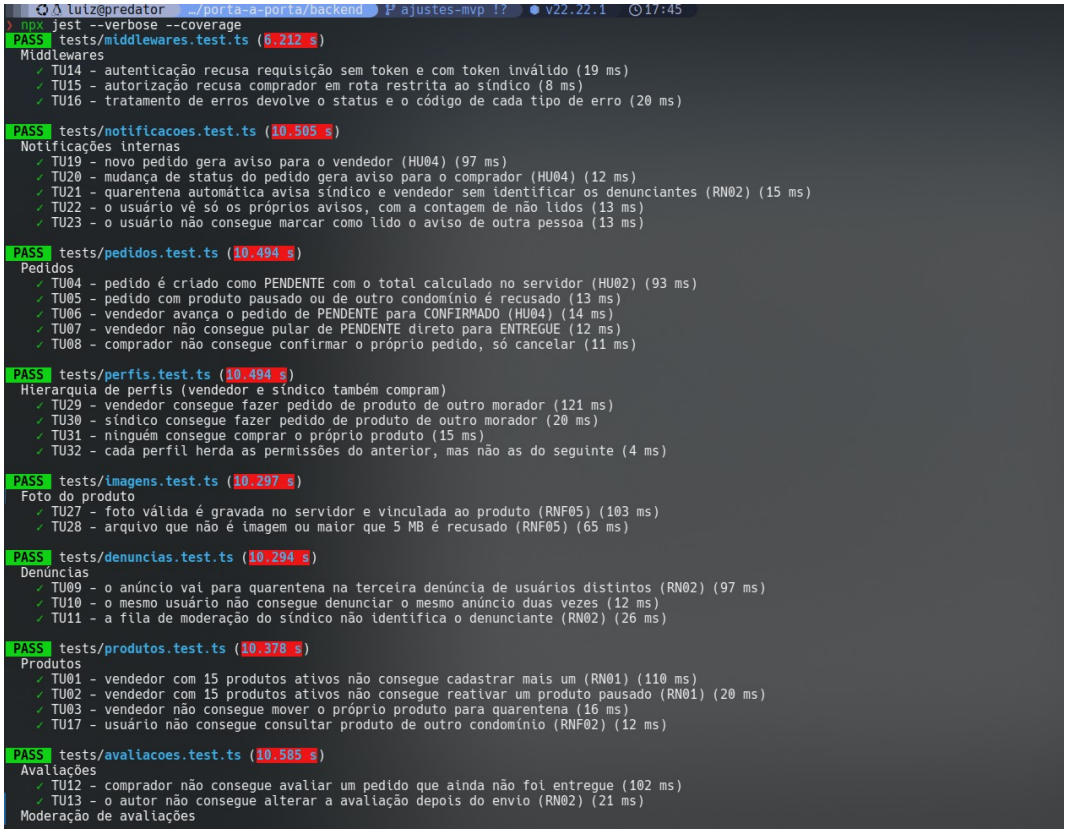
ID	Unidade testada	Cenário	Resultado esperado	Resultado obtido
TU16	Tratamento de erros (errorHandler)	Erro de validação, registro duplicado e erro inesperado.	Status 400 (VALIDATION_ERROR, com a lista de campos), 409 (DUPLICATE_ENTRY) e 500 (INTERNAL_ERROR), respectivamente.	Sucesso
TU17	Consulta de produto (GET /produtos/:id)	Usuário consulta um produto de outro condomínio (RNF02).	Status 404 e código PRODUTO_NOT_FOUND.	Sucesso
TU18	Carrinho de compras (cartStore)	O mesmo produto é adicionado duas vezes e depois tem a quantidade alterada para zero.	As quantidades são somadas e os totais recalculados; ao zerar, o item sai do carrinho.	Pendente
TU19	Notificações (POST /pedidos)	Comprador faz um pedido (HU04).	O vendedor recebe um aviso de novo pedido, com referência ao pedido.	Sucesso
TU20	Notificações (PATCH /pedidos/:id/status)	Vendedor muda o status do pedido (HU04).	O comprador recebe um aviso com o novo status.	Sucesso
TU21	Notificações (POST /denuncias)	Terceira denúncia coloca o anúncio em quarentena (RN02).	Síndico e vendedor são avisados; o aviso não identifica os denunciantes.	Sucesso
TU22	Listagem de avisos (GET /notificacoes)	Usuário consulta os próprios avisos.	Retorna só os avisos do usuário, com a contagem de não lidos.	Sucesso
TU23	Leitura de aviso (PATCH /notificacoes/:id/lida)	Usuário tenta marcar como lido o aviso de outra pessoa.	Status 404; o aviso continua não lido.	Sucesso
TU24	Remoção de avaliação (DELETE /avaliacoes/:id)	Síndico remove uma avaliação com justificativa (RN02).	Avaliação marcada como removida e log de auditoria REMOVER_AVALIACAO gravado com a justificativa.	Sucesso
TU25	Remoção de avaliação (DELETE /avaliacoes/:id)	Remoção sem justificativa ou de avaliação de outro condomínio.	Status 400 (VALIDATION_ERROR) e 404, respectivamente; nada é alterado.	Sucesso
TU26	Listagem de avaliações (GET /avaliacoes)	Vendedor consulta as avaliações recebidas (RN02).	A resposta não contém nome, unidade ou identificador de quem avaliou.	Sucesso
TU27	Foto do produto (PUT /produtos/:id/imagem)	Dono envia uma foto JPEG válida (RNF05).	Arquivo gravado na pasta de uploads e imagem_url atualizada no produto.	Sucesso
TU28	Foto do produto (PUT /produtos/:id/imagem)	Envio de arquivo que não é imagem e de foto maior que 5 MB (RNF05).	Status 415 (FORMATO_INVALIDO) e 413 (ARQUIVO_MUITO_GRANDE); nada é gravado.	Sucesso
TU29	Criação de pedido (POST /pedidos)	Vendedor compra produto de outro morador.	Pedido criado (201), pois o vendedor herda as permissões do comprador.	Sucesso
TU30	Criação de pedido (POST /pedidos)	Síndico compra produto de outro morador.	Pedido criado (201), pois o síndico herda as permissões do comprador.	Sucesso
TU31	Criação de pedido (POST /pedidos)	Usuário tenta comprar o próprio produto.	Status 400 e código PRODUTO_PROPRIO; nenhum pedido é criado.	Sucesso
TU32	Hierarquia de perfis (temPerfil)	Verificação das permissões de cada perfil.	Cada perfil tem as permissões do anterior, mas não as do seguinte.	Sucesso
TU33	Pedido de participação (POST /vinculos)	Morador pede para participar de outro condomínio como vendedor.	Pedido gravado como PENDENTE e aviso para o síndico daquele condomínio.	Sucesso
TU34	Middleware de autenticação e troca (POST /vinculos/trocar)	Acesso a outro condomínio com o pedido ainda pendente (RNF02).	Status 403 e código VINCULO_INVALIDO; nenhum dado do outro condomínio é lido e nenhum token é emitido.	Sucesso

ID	Unidade testada	Cenário	Resultado esperado	Resultado obtido
TU35	Decisão do síndico (PATCH /vinculos/:id)	Síndico aprova o pedido definindo o perfil.	Vínculo APROVADO com o perfil escolhido, log APROVAR_VINCULO e aviso no condomínio de origem.	Sucesso
TU36	Decisão do síndico (PATCH /vinculos/:id)	Recusa sem motivo e pedido de outro condomínio.	Status 400 e 404, respectivamente; nada é alterado nem registrado.	Sucesso
TU37	Vitrine (GET /produtos)	Usuário com vínculo aprovado vê o outro condomínio.	Lista os produtos do condomínio ativo e mostra a unidade do vínculo do vendedor.	Sucesso
TU38	Criação de produto (POST /produtos)	Vendedor aprovado só como comprador no outro condomínio tenta anunciar.	Status 403; o perfil vale o que o síndico aprovou naquele condomínio.	Sucesso
TU39	Painel de vendas (GET /pedidos/dashboard)	Vendedor com pedidos entregues, em preparo e cancelados.	Totais somam só os itens do vendedor; cancelados ficam fora do vendido e dos mais vendidos.	Sucesso
TU40	Painel de vendas (GET /pedidos/dashboard)	Comprador tenta abrir o painel.	Status 403; os pedidos não são consultados.	Sucesso
TU41	Alteração de status do produto (PATCH /produtos/:id/status)	Vendedor tenta reativar ou pausar o próprio anúncio em quarentena (RN02).	Status 403 e código PRODUTO_EM_QUARENTENA; o status não muda.	Sucesso
TU42	Vitrine e detalhe do produto (GET /produtos)	Outro morador procura um anúncio em quarentena (RN02).	O anúncio não aparece na vitrine e o detalhe responde 404.	Sucesso
TU43	Detalhe do produto (GET /produtos/:id)	Vendedor dono e síndico abrem o anúncio em quarentena.	O vendedor recebe só o motivo do síndico ou, sem decisão dele, o da suspensão automática, sem os motivos das denúncias; o síndico recebe também os motivos, sem identificação.	Sucesso
TU44	Decisão de quarentena (PATCH /denuncias/:produtoId/quarentena)	Síndico coloca um anúncio ativo em quarentena sem motivo e depois com motivo.	Sem motivo, status 400; com motivo, registro na auditoria e aviso ao vendedor contendo o motivo.	Sucesso
TU45	Decisão de quarentena (PATCH /denuncias/:produtoId/quarentena)	Síndico mantém sem motivo um anúncio já em quarentena por denúncias.	Aceito; a auditoria e o aviso ao vendedor usam o motivo automático.	Sucesso
TU46	Foto do produto (PUT /produtos/:id/imagem)	Foto enviada em multipart/form-data, como o app envia.	A foto é gravada igual ao original e vinculada ao produto.	Sucesso
TU47	Foto de perfil (PUT /usuarios/me/foto)	Usuário envia a própria foto.	Arquivo gravado e foto_url atualizada no usuário.	Sucesso
TU48	Painel de usuários (GET /usuarios/painel)	Síndico consulta o painel do condomínio.	Resumo com novos, suspensos e advertidos; lista sem e-mail nem CPF.	Sucesso
TU49	Sanções (POST /usuarios/:id/sancoes)	Síndico suspende um vendedor por 7 dias; antes, tenta sem informar os dias.	Sem dias, status 400; com dias, sanção com data de fim, anúncios pausados, auditoria e aviso.	Sucesso
TU50	Middleware de autenticação e login	Usuário suspenso acessa a vitrine; usuário banido faz login.	Status 403 USUARIO_SUSPENSO com o motivo e 403 USUARIO_BANIDO sem token.	Sucesso
TU51	Sanções (POST /usuarios/:id/sancoes)	Síndico tenta punir a si mesmo e quem não é do condomínio; comprador tenta punir.	Status 400, 404 e 403, respectivamente; nenhuma sanção criada.	Sucesso
TU52	Reativação (POST /usuarios/:id/reativar)	Síndico reativa um usuário suspenso.	Sanção encerrada, log REATIVAR_USUARIO e aviso ao usuário.	Sucesso

ID	Unidade testada	Cenário	Resultado esperado	Resultado obtido
TU53	Foto do produto (PUT /produtos/:id/imagem)	JPEG informado pelo aparelho como "image/jpg" ou sem tipo.	Aceito, pois o formato é verificado pelo conteúdo do arquivo.	Sucesso
TU54	Criação de avaliação (POST /avaliacoes)	Comprador avalia um pedido entregue com dois produtos: sem informar o produto, um produto de fora do pedido, o segundo produto e o mesmo produto de novo.	Status 400 (PRODUTO_OBRIGATORIO e PRODUTO_FORA_DO_PEDIDO), 201 com o produto gravado e 409; o vendedor recebe o aviso NOVA_AVALIACAO sem o nome de quem avaliou.	Sucesso
TU55	Avaliações do produto (GET /avaliacoes/produto/:id)	Morador consulta as avaliações de um produto com notas 5, 4 e 5.	Média 4,7, total 3 e quantidade por nota; só avaliações do produto, do condomínio e não removidas, sem pedido nem autor.	Sucesso
TU56	Fila de moderação (GET /denuncias/produtos)	Síndico consulta a fila com 3 denúncias de um anúncio em quarentena e 1 de um anúncio ativo (RN02).	Um item por anúncio, quarentena primeiro, com a quantidade e os motivos; resumo 1 em quarentena e 1 na vitrine; sem identificação do denunciante.	Sucesso
TU57	Histórico de moderação (GET /auditoria)	Síndico consulta o histórico escondendo as consultas de denúncias.	Cada registro traz o nome do anúncio ou do usuário afetado; a consulta exclui VISUALIZAR_DENUNCIAS e vale só para o condomínio ativo.	Sucesso
TU58	Histórico de moderação (GET /auditoria)	Comprador tenta abrir o histórico.	Status 403; o log não é consultado.	Sucesso

Indicador	Resultado
Quantidade de testes executados	57
Testes com sucesso	57
Testes com falha	0
Taxa de sucesso	100%
Cobertura de código, quando disponível	80,00% das linhas (rotas, serviços e middlewares do back-end)

Evidências





3.3 Interfaces com usuário

Esta seção não precisa apresentar novamente todas as telas implementadas, pois a interface será avaliada diretamente no software em execução.

Item	Informação
Situação da implementação	Concluída
Principais telas implementadas	Acesso: boas-vindas, login, cadastro e escolha do perfil e do condomínio. Abas: vitrine (grade de produtos com foto, filtro por categoria e o condomínio exibido no topo), carrinho (com escolha do tipo de entrega, do dia e do período), pedidos (compras e vendas), painel de vendas e anúncios do vendedor, moderação, para o síndico, em quatro abas (denúncias agrupadas por anúncio com a decisão de quarentena, avaliações com remoção justificada, usuários com pedidos de participação e sanções, e histórico das decisões) e perfil (com foto). Demais telas: busca (aberta pelo campo da vitrine), condomínios (troca do condomínio ativo e pedido de participação), detalhe do produto (com

	foto, nota média, distribuição das notas e comentários anônimos) e cadastro de produto, detalhe do pedido com a linha do tempo do status e a avaliação de cada produto depois da entrega, avisos (notificações internas) e configurações (privacidade, ajuda e sobre).

Durante a apresentação, o grupo deverá demonstrar as interfaces funcionando no sistema.

3.4 Banco de dados

O banco de dados foi implementado em PostgreSQL, executado em um container Docker com volume persistente. A estrutura é definida no schema do Prisma e criada por migrações versionadas no repositório. Os identificadores são do tipo UUID e todas as entidades de negócio carregam o condominio_id, que isola os dados de cada condomínio (RNF02).

Item	Informação
SGBD utilizado	PostgreSQL 16, em container Docker, acessado pela API por meio do Prisma ORM.
Situação da implementação	Concluída
Local dos scripts no repositório	https://github.com/luizgaf/porta-a-porta, nas pastas backend/prisma (schema, migrações e dados iniciais) e docker (container do banco e scripts de execução).

Os scripts utilizados em cada etapa são os seguintes:
• Criação do banco: docker/docker-compose.yml, que cria o container, o usuário e o banco porta_a_porta, e docker/init.sql, executado na primeira inicialização para habilitar as extensões usadas.
• Criação das tabelas: backend/prisma/schema.prisma (modelos, enums, chaves e índices) e a migração backend/prisma/migrations/20260922011543_init, com o script SQL que cria as tabelas.
• Atualização da estrutura: a migração 20261014180000_notificacoes_imagem_produto, que cria a tabela de notificações, acrescenta a coluna imagem_url aos produtos e remove a coluna push_token, que não é mais usada, e a migração 20261019120000_vinculos_condominios, que cria a tabela de vínculos com outros condomínios, a migração 20261027150000_foto_perfil_sancoes, que acrescenta a foto de perfil e cria a tabela de sanções, e a migração 20261109160000_avaliacao_por_produto, que liga cada avaliação a um produto e copia as avaliações antigas para cada produto do pedido. As migrações são aplicadas com o comando prisma migrate deploy.
• Inserção de dados: backend/prisma/seed.ts, executado com npm run prisma:seed, junto com as fotos da pasta backend/prisma/seed-images.
• Execução: docker/run.sh setup sobe o banco, aplica as migrações e insere os dados iniciais; docker/run.sh reset apaga o volume e recria o banco do zero.

Dados iniciais

Sim. O seed cria os dados necessários para executar e demonstrar a aplicação:
• 10 condomínios de teste (seed-cond-1 a seed-cond-10), cada um com o seu síndico (sindico1@pap.com a sindico10@pap.com, senha sindico123);
• compradores e vendedores de teste (senhas comprador123 e vendedor123): no primeiro condomínio, comprador1@pap.com (Apto 101), comprador2@pap.com (Apto 102) e vendedor1@pap.com; nos condomínios 3 a 10, um comprador (comprador3@pap.com a comprador10@pap.com); nos condomínios 2 a 10, um vendedor (vendedor2@pap.com a vendedor10@pap.com);
• 15 produtos do vendedor do primeiro condomínio, em 8 categorias e com foto, sendo um deles em quarentena com as 3 denúncias que a justificam, e um produto copiado deles para cada vendedor dos condomínios 2 a 10;
• 4 pedidos já entregues de comprador1 e comprador2 no primeiro condomínio, com as avaliações de três produtos, para o anúncio e a vitrine mostrarem as notas.

Avisos, sanções e as demais denúncias e avaliações não são pré-cadastrados: eles são gerados durante a demonstração, a partir das contas de teste. O seed pode ser executado mais de uma vez sem duplicar registros, pois atualiza os dados já existentes.

---

4. TESTES

4.1 Teste funcional

Os testes funcionais trataram o sistema como caixa-preta: cada cenário foi executado em 27/10/2026 (os cenários TF05, TF07 e TF14 foram refeitos ou incluídos em 09/11/2026, depois das mudanças na quarentena, nas avaliações e na moderação) contra a API em execução, com um banco PostgreSQL novo preenchido pelo seed, enviando as mesmas requisições que o aplicativo envia e observando apenas as respostas, os avisos gerados e os dados gravados. Além das quatro histórias de usuário, foram testadas as regras de negócio e os requisitos não funcionais verificáveis.

ID	Requisito/História	Cenário de teste	Resultado esperado	Resultado obtido	Situação
TF01	HU01	Comprador consulta a vitrine do condomínio e filtra pela categoria Bebidas.	Aparecem só anúncios ativos ou pausados do condomínio do comprador; o filtro mantém apenas a categoria escolhida.	14 anúncios do condomínio 1 (ativos e pausados); filtro Bebidas devolveu 3 anúncios, todos da categoria Bebidas	Aprovado
TF02	HU02	Comprador finaliza um pedido com dois produtos, entrega na portaria e janela "Qua, 28/10 - Tarde (12h-18h)".	Pedido criado como pendente, com total calculado pelo servidor, janela gravada e aviso ao vendedor.	Pedido criado (201) como PENDENTE, total R$ 19,00 (esperado R$ 19,00), janela gravada "Qua, 28/10 - Tarde (12h-18h)" e aviso NOVO_PEDIDO para o vendedor	Aprovado
TF03	HU03 / RN01	Vendedor cadastra anúncios até ultrapassar 15 ativos.	Os anúncios são aceitos até 15 ativos; o seguinte é recusado com a mensagem do limite.	Com 12 ativos, 3 anúncios foram aceitos até chegar a 15; o seguinte foi recusado com 400 LIMIT_EXCEEDED	Aprovado
TF04	HU04	Vendedor tenta entregar um pedido pendente e depois o leva por todas as etapas.	O pulo de etapa é recusado; cada etapa válida é aceita e gera aviso ao comprador.	Pulo de PENDENTE para ENTREGUE recusado (400); as 4 etapas seguintes aceitas (200 em cada uma); o comprador recebeu 4 avisos de mudança de status	Aprovado
TF05	RN02 (a)	Três moradores diferentes denunciam o mesmo anúncio.	O anúncio entra em quarentena e some para os demais moradores; o vendedor o vê só com o motivo do síndico, sem os motivos das denúncias, e não consegue reativá-lo; o	As 3 denúncias foram aceitas (201) e o anúncio foi para QUARENTENA no banco; o comprador recebeu 404 no detalhe; o vendedor vê o motivo "Muitas	Aprovado

			síndico vê os motivos.	denúncias: produto sob análise do síndico." e nenhum dos motivos das denúncias, e não consegue reativá-lo (403); o síndico vê os 3 motivos, sem identificação	
TF06	RN02 (a, c)	Síndico mantém sem motivo o anúncio já suspenso, tenta suspender sem motivo um anúncio ativo, registra um motivo e consulta a fila de denúncias.	Manter dispensa o motivo; suspender um anúncio ativo exige motivo; com motivo, o vendedor recebe o aviso; a fila não identifica os denunciantes.	Manter o anúncio já suspenso sem motivo foi aceito (200); suspender um anúncio ativo sem motivo foi recusado (400); com motivo, a decisão foi aceita (200), o vendedor recebeu o aviso e vê o motivo; a fila de denúncias não identifica os denunciantes	Aprovado
TF07	RN02 (b)	Comprador avalia cada produto de um pedido entregue com dois produtos, tenta repetir e editar; outro morador consulta o anúncio; síndico remove uma avaliação.	Uma avaliação por produto, só depois da entrega; repetição e edição recusadas; o anúncio mostra média e comentários sem o autor; o vendedor é avisado; a remoção exige justificativa, sai da média e fica na auditoria.	Antes da entrega, recusada (404); sem informar o produto, 400 PRODUTO_OBRIGATORIO; os dois produtos avaliados (201 e 201); repetição recusada (409), edição recusada (403) e outro comprador recusado (403); o anúncio mostrou média 5,0 com 1 avaliação, só com nota, comentário e data, e a vitrine a mesma nota; o vendedor viu as 2 avaliações sem pedido nem autor e recebeu 2 avisos de nova avaliação; remoção sem justificativa recusada (400) e com justificativa aceita (200); o produto ficou sem avaliações visíveis e a auditoria registrou REMOVER_AVALIACAO com a justificativa	Aprovado
TF08	RNF02	Morador do condomínio 3 consulta a vitrine, tenta abrir anúncio do condomínio 1 e trocar para ele; acesso sem login.	Acesso negado ao anúncio e à troca de condomínio; sem login, a API recusa a requisição.	Morador do condomínio 3 vê só a vitrine do condomínio 3, não abre anúncio do condomínio 1 (404) nem troca para ele sem aprovação (403); sem token a API responde 401	Aprovado
TF09	RNF05	Vendedor envia uma foto JPEG do jeito que o app envia (formulário, tipo "image/jpg"), um arquivo que não é imagem e um maior que 5 MB.	Só a foto válida é aceita; os outros dois são recusados.	Foto JPEG de 99 KB enviada como o app envia (multipart, tipo "image/jpg") aceita (200); arquivo que não é imagem recusado (415); arquivo acima de 5 MB recusado (413)	Aprovado
TF10	Vários condomínios	Vendedor do condomínio 2 pede para participar do condomínio 1; o síndico aprova apenas como comprador.	Após a aprovação ele troca para o condomínio 1 com a unidade informada, vê a vitrine, mas não consegue anunciar lá.	Vendedor do condomínio 2 pediu participação no condomínio 1 (201); o síndico aprovou só como comprador (200);	Aprovado

				a troca deu acesso à vitrine do condomínio 1 com a unidade Apto 905, mas anunciar foi recusado (403)	
TF11	Painel de vendas	Vendedor abre o painel antes e depois de um novo pedido; comprador tenta abrir o painel.	O painel mostra o total vendido e o valor em andamento corretos; o comprador não tem acesso.	Painel do vendedor mostra R$ 19,00 vendidos (o pedido entregue) e, após novo pedido, R$ 12,00 em andamento; o comprador não acessa o painel (403)	Aprovado
TF12	Moderação de usuários	Síndico abre o painel de usuários, adverte e suspende por 3 dias uma compradora, que tenta entrar; depois a reativa. Síndico tenta punir a si mesmo.	Painel sem e-mail ou CPF; a compradora suspensa não entra e vê o motivo; após a reativação, entra; o síndico não pune a si mesmo.	O painel listou 6 usuários (6 novos), sem e-mail; advertência (201) e suspensão de 3 dias (201) aplicadas; a compradora suspensa teve o login recusado (403) com o motivo; após a reativação (200) o login voltou a funcionar (200); o síndico não conseguiu punir a si mesmo (400)	Aprovado
TF13	Foto de perfil	Comprador envia uma foto de perfil e depois a remove.	A foto é gravada e aparece no perfil; ao remover, o perfil fica sem foto.	Foto de perfil enviada (200) e devolvida no perfil (/uploads/505833f2-7232-4f41-9318-ad1ad3754f2e.jpg); ao remover (200), o perfil voltou sem foto	Aprovado
TF14	RN02 (moderação)	Síndico abre a fila de denúncias, restaura um anúncio e consulta o histórico; comprador tenta abrir a fila e o histórico.	A fila mostra um item por anúncio, quarentena primeiro, sem identificar denunciantes; a restauração atualiza a fila; o histórico mostra a decisão com o nome do anúncio e sem as consultas; o comprador não acessa.	A fila listou primeiro "Banana Prata" em quarentena com 3 denúncias (resumo: 2 em quarentena), sem identificar quem denunciou; após restaurar (200), o resumo passou a 1 em quarentena e 1 na vitrine; o histórico mostrou "RESTAURAR_PRODUTO - Banana Prata" com a justificativa e a remoção de avaliação "3 estrelas em Suco de Laranja Natural"; a consulta de denúncias gravada no log não apareceu no histórico; o comprador recebeu 403 na fila e no histórico	Aprovado

Indicador	Resultado
Quantidade total de testes	14
Testes aprovados	14
Testes reprovados	0
Taxa de sucesso	100%

4.2 Teste de integração

4.2.1 Integração com banco de dados

Cada operação foi feita pela API e conferida por consulta SQL direta no mesmo banco PostgreSQL, na mesma execução dos testes funcionais.

ID	Integração testada	Operação	Resultado esperado	Resultado obtido
TI01	Aplicação ↔ Banco (pedidos e itens)	Inserção	O pedido grava uma linha em pedidos e uma por produto em itens_pedido, com o preço do momento (RN03).	POST /pedidos gravou 1 linha em pedidos (valor_total 19,00) e 2 em itens_pedido (quantidades somando 3, preços 3,50 e 12,00)
TI02	Aplicação ↔ Banco (produtos)	Consulta	A vitrine devolve os mesmos anúncios que a consulta SQL do condomínio e dos status permitidos.	GET /produtos devolveu 16 anúncios, o mesmo número da consulta direta no banco (16) para o condomínio 1
TI03	Aplicação ↔ Banco (produtos e itens)	Atualização	A alteração de preço é gravada no produto sem mudar o preço congelado dos pedidos anteriores.	PUT /produtos atualizou o preço no banco para 8,50; o item do pedido anterior manteve 3,50
TI04	Aplicação ↔ Banco (produtos)	Exclusão	A exclusão é lógica: a linha permanece com status EXCLUIDO e o anúncio sai da vitrine.	DELETE /produtos manteve a linha no banco com status EXCLUIDO (exclusão lógica, preservando pedidos antigos) e o anúncio saiu da vitrine
TI07	Aplicação ↔ Banco (avaliações)	Inserção e consulta	Cada produto avaliado grava uma linha com o produto; o banco recusa uma segunda avaliação do mesmo produto no mesmo pedido; a média da API é igual à calculada no banco.	As duas avaliações do TF07 gravaram 2 linhas em avaliacoes, uma por produto (notas 5 e 3); uma inserção repetida direto no banco foi recusada pela restrição avaliacoes_pedido_id_produto_id_key; média do Pão de Queijo: 5,0 com 1 avaliação no banco e na API
TI08	Migração ↔ Banco (avaliações antigas)	Atualização	Uma avaliação antiga de um pedido com dois produtos passa a valer para cada produto, sem perder nota e comentário.	Num banco com a estrutura anterior, uma avaliação (nota 5, "bom") de um pedido com dois produtos virou 2 linhas, uma para cada produto, com a mesma nota e o mesmo comentário, após aplicar a migração 20261109160000_avaliacao_por_produto

4.2.2 Integração com APIs e serviços

O projeto não usa APIs nem serviços de terceiros. Foram testados os dois componentes com que a API se integra além do banco: o armazenamento das fotos no servidor e a autenticação por token usada pelo aplicativo.

ID	Componente	Integração	Resultado esperado	Resultado obtido
TI05	Armazenamento de fotos (pasta uploads)	Envio da foto pela API, gravação no disco e acesso pelo endereço /uploads.	O arquivo é gravado na pasta de uploads e servido pela API com o mesmo conteúdo enviado.	A foto enviada foi gravada na pasta de uploads (abd7e6c1-ae11-43cd-8b28-5be51fe48ed6.jpg) e servida pela API em /uploads/abd7e6c1-ae11-43cd-8b28-5be51fe48ed6.jpg (200, image/jpeg, 101619 bytes, igual ao enviado)
TI06	Autenticação JWT (aplicativo ↔ API)	Login, uso do token nas requisições e recusa de token inválido.	O token identifica o usuário e o condomínio; um token inválido é recusado.	Login devolveu o token JWT; com ele, GET /auth/me identificou o usuário e o condomínio; token inválido recebeu 401

4.3 Teste de usabilidade - Heurísticas de Nielsen

A avaliação percorreu as telas do aplicativo nos perfis de comprador, vendedor e síndico. As telas de acesso e a vitrine já seguem o novo visual; a moderação e as avaliações foram refeitas com o novo visual; carrinho, pedidos e perfil ainda usam o visual anterior, o que explica as heurísticas marcadas como parciais.

Heurística	Atende?	Evidência/Justificativa
1. Visibilidade do status do sistema	Sim	O pedido mostra o status com cor e linha do tempo; anúncios têm selos ("Indisponível", "Em quarentena"); o sino da vitrine mostra os avisos não lidos; o topo da vitrine mostra o condomínio exibido; carregamentos têm indicador e as listas atualizam ao puxar; as abas da moderação mostram quantos anúncios estão em quarentena e quantos pedidos de participação aguardam.
2. Correspondência entre sistema e mundo real	Sim	O vocabulário é o do condomínio: unidade, portaria, síndico, vizinho. A unidade aparece como etiqueta de porta ("Apto 101"), a entrega é escolhida por dia e período (manhã, tarde, noite) e os valores estão em reais.
3. Controle e liberdade do usuário	Parcial	O carrinho é editável, formulários e janelas têm "Cancelar", o pedido pode ser cancelado antes da entrega, a foto de perfil pode ser trocada ou removida e o síndico pode desfazer suspensões e banimentos. Por regra de negócio (RN02), a avaliação não pode ser editada depois de enviada.
4. Consistência e padrões	Parcial	Componentes e tokens de design são compartilhados e as cores de status são as mesmas na lista de pedidos e no painel de vendas. As telas ainda não refeitas usam cores e tipografia diferentes das telas novas.
5. Prevenção de erros	Sim	Confirmação antes de excluir; aviso do limite de 15 anúncios; só aparecem períodos de entrega que ainda não terminaram; total calculado no servidor; ninguém compra o próprio produto; foto validada por formato e tamanho; motivo

		obrigatório nas decisões do síndico.
6. Reconhecimento em vez de memorização	Sim	Categorias em chips, abas com ícone e nome, seletores de dia e período em vez de texto livre, unidade de entrega já preenchida com a do usuário e avisos que abrem direto o pedido ou o anúncio.
7. Flexibilidade e eficiência de uso	Parcial	Há busca por texto, filtros por categoria e por status dos anúncios, e atalhos a partir dos avisos. Não há recursos para usuários frequentes, como repetir um pedido ou favoritar vendedores.
8. Design estético e minimalista	Parcial	As telas de acesso, a vitrine e a moderação foram simplificadas (grade de produtos, menos emojis, tipografia própria). A moderação passou a mostrar um cartão por anúncio, e não um por denúncia. Carrinho, pedidos e perfil ainda têm mais elementos e estilos do visual anterior.
9. Reconhecimento, diagnóstico e recuperação de erros	Sim	As mensagens de erro da API estão em português e dizem o que fazer (ex.: "Limite de 15 produtos ativos atingido. Pause ou exclua um produto antes de ativar outro."); erros de formulário aparecem junto aos campos; o painel de vendas e as abas da moderação oferecem "Tentar de novo"; o vendedor vê em destaque o motivo da quarentena dado pelo síndico; o usuário suspenso ou banido vê o motivo e o prazo ao tentar entrar.
10. Ajuda e documentação	Parcial	A tela Ajuda traz perguntas frequentes sobre perfil de vendedor, quarentena, entrega, cancelamento, avaliação, limite de anúncios e outros condomínios, e a tela Privacidade explica as regras de anonimato. Não há ajuda contextual dentro das demais telas.

Problemas de usabilidade encontrados

ID	Problema identificado	Heurística relacionada	Correção realizada
PU01	Na vitrine, os cards em carrossel tinham tamanhos diferentes e deixavam espaço em branco embaixo.	8. Design estético e minimalista	Vitrine refeita em grade de duas colunas, com cards de mesmo tamanho.
PU02	Em celulares Android com botões de navegação, a barra de abas ficava atrás desses botões.	4. Consistência e padrões	A barra de abas passou a respeitar a área segura da tela.
PU03	Botões lado a lado (como Pausar/Ativar e Excluir no painel Vender) apareciam cortados, com só a ponta do segundo visível.	8. Design estético e minimalista	Os botões passaram a dividir a largura da linha.
PU04	A janela de entrega era digitada livremente, permitindo horários vagos ou já passados (HU02).	5. Prevenção de erros	Seletores de dia (hoje ou amanhã) e período, mostrando só períodos ainda disponíveis.
PU05	O vendedor não sabia por que o anúncio foi suspenso, e a mensagem de quarentena da página do produto nunca aparecia.	1. Visibilidade do status do sistema	O motivo da quarentena é exibido ao vendedor e ao síndico, e a mensagem foi corrigida.

PU06	A tela de privacidade tinha interruptores sem efeito e botões "Em breve".	2. Correspondência entre sistema e mundo real	Substituída por uma explicação das regras de privacidade aplicadas pelo sistema.
PU07	A Ajuda citava um chat inexistente e regras de cancelamento diferentes das reais.	10. Ajuda e documentação	Respostas reescritas de acordo com as regras do sistema, com perguntas sobre quarentena e outros condomínios.
PU08	O motivo da quarentena aparecia no meio de uma frase, sem as denúncias que a causaram.	1. Visibilidade do status do sistema	Quadro próprio com o motivo em destaque. Na v9, os motivos escritos pelos denunciantes passaram a aparecer só para o síndico, para preservar o anonimato (RN02).
PU09	Fotos JPEG eram recusadas como "formato inválido" em alguns celulares Android.	9. Reconhecimento, diagnóstico e recuperação de erros	Envio da foto como formulário e formato verificado pelo conteúdo do arquivo, e não pelo tipo informado pelo aparelho.
PU10	Na tela do pedido entregue, o botão "Avaliar" não abria nada, e um pedido com vários produtos recebia uma única nota.	1. Visibilidade do status do sistema	Botão "Avaliar" ao lado de cada produto, com janela de estrelas e comentário; a nota aparece em seguida no pedido.
PU11	As avaliações não apareciam no anúncio, então o comprador não tinha como saber a opinião dos vizinhos.	6. Reconhecimento em vez de memorização	Anúncio com média, distribuição das notas e comentários anônimos; a vitrine mostra a nota sobre a foto.
PU12	A fila de moderação repetia o mesmo anúncio uma vez para cada denúncia.	8. Design estético e minimalista	Um cartão por anúncio, com foto, vendedor, quantidade de denúncias e o motivo mais recente, e filtros por situação.
PU13	Na moderação, falhas ao carregar só apareciam no console, e o síndico não tinha como ver as decisões já tomadas.	9. Reconhecimento, diagnóstico e recuperação de erros	Mensagem na própria tela com "Tentar de novo" e aba Histórico com as decisões e justificativas.

5. SITUAÇÃO FINAL DO MVP

O MVP entregue é um aplicativo móvel em React Native (Expo) que se comunica com uma API REST em Node.js/Express e um banco PostgreSQL. Todas as funcionalidades previstas no escopo (seção 1.1.3) foram implementadas e podem ser demonstradas com as contas de teste criadas pelo seed. Durante o desenvolvimento, o escopo ganhou quatro itens que também estão funcionando: participação em mais de um condomínio, painel de vendas, moderação de usuários com sanções e foto de perfil.

Como definido no escopo, ficaram de fora o pagamento online, a logística de entrega externa e as notificações push. O aplicativo ainda não foi publicado nas lojas e a API não está hospedada em um servidor: a demonstração é feita com o Expo Go no celular e a API e o banco executando no computador, com o banco em Docker.

5.1 Funcionalidades previstas x implementadas

Funcionalidade	Prevista	Implementada	Funcionando	Observação
Cadastro, login e perfis (comprador, vendedor e síndico)	Sim	Sim	Sim	Senha guardada com hash e acesso por token JWT; perfis com herança; dados isolados por condomínio (RNF02).
Vitrine digital com fotos, busca e categorias	Sim	Sim	Sim	Limite de 15 anúncios ativos por vendedor (RN01); foto de até 5 MB guardada no servidor (RNF05).
Carrinho e pedido vinculado à unidade	Sim	Sim	Sim	Valor total calculado pela API com o preço do banco; preço congelado em cada item do pedido.
Gerenciador de status do pedido	Sim	Sim	Sim	Transições validadas pela API; o vendedor conduz o pedido e o comprador só pode cancelar.
Notificações internas no aplicativo	Sim	Sim	Sim	Avisos de novo pedido, mudança de status, quarentena, avaliação e participação, vistos ao abrir ou atualizar o app.

Denúncias e quarentena de anúncios	Sim	Sim	Sim	Quarentena automática com 3 denúncias; o vendedor vê só o motivo dado pelo síndico (RN02).
Avaliações dos produtos	Sim	Sim	Sim	Uma por produto de pedido entregue; média e comentários exibidos no anúncio sem identificar quem avaliou.
Painel do síndico: moderação, sanções e histórico	Sim	Sim	Sim	Fila de denúncias, remoção de avaliações, advertência, suspensão e banimento; decisões registradas na auditoria. As sanções foram incluídas no escopo durante o desenvolvimento.
Participação em mais de um condomínio	Sim	Sim	Sim	Incluída no escopo durante o desenvolvimento; o síndico do outro condomínio aprova ou recusa o pedido.
Painel de vendas do vendedor	Sim	Sim	Sim	Incluído no escopo durante o desenvolvimento; total vendido, pedidos por status, mais vendidos e últimos 6 meses.
Foto de perfil	Sim	Sim	Sim	Incluída no escopo durante o desenvolvimento. O envio de fotos funciona no celular, mas não na versão para navegador.
Pagamento online	Não	Não	-	Fora do escopo: o pagamento é combinado diretamente entre os vizinhos.
Notificações push	Não	Não	-	Fora do escopo: exige build nativo do aplicativo e serviço externo.
Logística de entrega externa	Não	Não	-	Fora do escopo: as entregas acontecem dentro do condomínio.

5.2 Persistência dos dados

Os dados são gravados no PostgreSQL 16, executado em um container Docker (docker/docker-compose.yml) com o volume postgres_data montado na pasta de dados do banco. Por isso, parar ou reiniciar o container, a API ou o computador não apaga nenhum registro: os dados só são removidos pelo comando docker/run.sh reset, que apaga o volume de propósito para recriar o banco do zero. As fotos dos produtos e de perfil ficam na pasta backend/uploads do servidor, e o banco guarda o endereço de cada arquivo.

No aplicativo, o AsyncStorage guarda a sessão (token e dados do usuário) e o carrinho, que continuam disponíveis quando o app é fechado e aberto de novo. O carrinho é só um rascunho: o pedido passa a existir quando é gravado no banco pela API.

Evidência: em 09/11/2026, às 22h18, o banco e a API foram iniciados depois de o ambiente ter sido desligado. As duas avaliações gravadas às 15h59, nos testes da tarde, continuavam no banco. Em seguida, um pedido novo foi criado pela API; a API e o banco foram encerrados e iniciados de novo, e o mesmo pedido foi encontrado, com o mesmo status, valor e itens, tanto pela API quanto por consulta SQL direta. No ambiente com Docker, o mesmo teste é feito com docker/run.sh restart.

```
$ date
09/11/2026 22:18:30

# 1. Inicia o banco (PostgreSQL 16) e a API
server started
API: {"status":"ok","timestamp":"2026-11-10T01:18:34.521Z"}

# 2. Dados gravados na sessão anterior (testes de 09/11/2026, tarde) continuam no banco
$ SELECT ... FROM avaliacoes JOIN produtos
09/11 15:59  Pão de Queijo  nota 5
09/11 15:59  Suco de Laranja Natural  nota 3

# 3. Cria um pedido novo pela API (comprador2)
POST /api/pedidos -> 201  id df9fe37f...  status PENDENTE  valor R$ 37,00

# 4. Encerra a API e o banco
server stopped
API: fora do ar

# 5. Inicia de novo e consulta o mesmo pedido
server started
GET /api/pedidos/df9fe37f... -> 200  status PENDENTE  valor R$ 37,00  itens [('Café Torrado Especial', 2)]
$ SELECT status, valor_total FROM pedidos WHERE id = 'df9fe37f...'
PENDENTE | 37.00
```

Figura: pedido criado, banco e API encerrados e iniciados de novo, e o pedido consultado em seguida.

5.3 Limitações conhecidas

A versão apresentada tem as seguintes limitações conhecidas:

- O aplicativo não foi publicado nas lojas e a API não está hospedada em servidor; a publicação (EAS Build, hospedagem da API e do banco) e a integração contínua ficaram como próximos passos.
- Os avisos internos aparecem quando o usuário abre ou atualiza o aplicativo, e não no mesmo instante, porque não há notificações push nem conexão em tempo real.
- Não há recuperação de senha pelo próprio usuário.
- Os testes automatizados cobrem só o back-end (57 testes de unidade, 80% das linhas); não há testes automatizados das telas nem testes de ponta a ponta, que estavam planejados.
- Na versão para navegador (Expo web), o envio de fotos não funciona, pois foi feito para o formato usado pelo celular; as demais telas funcionam no navegador.
- Carrinho, pedidos e perfil ainda usam o visual anterior ao novo design system, e as heurísticas avaliadas como parciais na seção 4.3 seguem como pontos de melhoria.
- As fotos ficam numa pasta do próprio servidor, sem cópia de segurança; numa hospedagem real precisariam de um serviço de armazenamento. A chave de assinatura dos tokens usada em desenvolvimento também precisa ser trocada antes da publicação.

6. CONSIDERAÇÕES FINAIS

O projeto atingiu o objetivo de oferecer uma vitrine digital exclusiva para cada condomínio, com pedidos vinculados à unidade e acompanhamento de status, tirando o comércio interno dos grupos de mensagens. O MVP vai além do

escopo inicial: o síndico modera anúncios, avaliações e usuários, o vendedor acompanha as vendas em um painel e um morador pode participar de mais de um condomínio.

Entre as decisões técnicas, as mais relevantes foram usar TypeScript no aplicativo e na API, versionar o banco com as migrações do Prisma e isolar os condomínios pelo condominio_id presente em todas as tabelas e no token de acesso. As regras que não podem depender do aplicativo ficaram na API: o valor do pedido é calculado com o preço do banco, as transições de status são validadas e a quarentena automática acontece no servidor. As notificações ficaram internas ao aplicativo, e não push, para não depender de build nativo e de serviço externo nesta fase.

As principais dificuldades foram o envio de fotos pelo celular, que exigiu três correções (no formato do arquivo, no tipo informado pelo aparelho e na forma de enviar o formulário), as atualizações do Expo durante o desenvolvimento e o equilíbrio da regra RN02: dar ao vendedor o motivo de uma quarentena sem revelar quem denunciou, e mostrar avaliações sem identificar quem avaliou. A participação em vários condomínios também exigiu revisar a autenticação, para que o perfil e a unidade valham por condomínio.

Como aprendizado, ficou o valor dos testes automatizados: os 57 testes de unidade e os testes funcionais e de integração executados pela API apontaram problemas antes da demonstração e deram segurança para mudar o código. Também ficou clara a importância de manter este documento coerente com o código a cada mudança, o que foi feito em versões sucessivas.

Como melhorias e funcionalidades futuras, o grupo considera publicar o aplicativo e a API, adicionar notificações push, pagamento por Pix, recuperação de senha, testes de ponta a ponta com integração contínua e o novo visual nas telas que ainda usam o anterior.

ANEXOS

Quando necessário, podem ser incluídos:

- diagramas em maior resolução;
- protótipos;
- scripts;
- evidências de testes;
- relatórios de cobertura;
- imagens complementares;
- documentação de APIs;
- demais evidências técnicas.

ORIENTAÇÕES PARA ENTREGA

- O documento deverá representar o que foi efetivamente desenvolvido pelo grupo.
- Não deverão ser apresentadas como concluídas funcionalidades que não estejam

implementadas no software.

- Todas as funcionalidades descritas como implementadas deverão poder ser demonstradas durante a apresentação do projeto.
- Diagramas, requisitos, banco de dados, interfaces e código deverão possuir coerência entre si.
- O código-fonte deverá estar disponível no repositório indicado no documento.
- O MVP deverá ser executável e permitir a demonstração das principais funcionalidades propostas.
- A persistência dos dados deverá ser demonstrada quando fizer parte dos requisitos da solução.
- Falhas, limitações ou itens não concluídos devem ser informados de forma explícita na seção “Situação Final do MVP”.

A existência apenas de documentação, protótipos, telas isoladas, scripts ou trechos de código não caracteriza a conclusão do projeto. Os diferentes componentes da solução deverão estar integrados e permitir a execução do fluxo principal do sistema.